

Modular and Parameterized CNN Accelerator with Systolic Convolution Arrays and Windowless Continuous Parallel Max-Pooling

The aim of this project, is to optimize the CNN network to the hardware level, facilitating data reuse, efficient data flow and reduction in ports and IO load for real world use.

To accomplish that , parallel processing, and data reuse were proposed. Suitable architecture was to be chosen or evolved from existing architectures.

Objectives :

1. To make convolution operation with systolic array facilitating parallel processing and data reuse
2. To integrate ReLU activation function
3. To make a max-pooling module without sliding window buffers, and with stride and pool size settings.
4. To connect the modules together and form a fully working, functional 3 Layer CNN for establishing proof of concept, proving modularity.

Methods used to make the system scalable, fully modular :

- Make every aspect of the system, like dimensions, bit-width, stride, pool size, parametrized.
- Use derived parameters using *localparam*
- Avoid hard-coding numbers.
- Test-bench logic shall also be parametrized, and test edge cases like mismatched dimensions, and ignoring extraneous feature map row and columns when input to max pool.
- Usage of Finite State Machines [FSM] to maintain precise timings, and making code readable.
- Avoiding vague flags and control signals excessively, since FSM is a better control logic method.
- Using genvar, generate - endgenerate and for loops to instantiate modules procedurally.
- Systolic array can be made scalable using the above method, otherwise, it is an extremely tedious affair to do so.

```
generate
  for (i = 0; i < SYS_SIZE; i = i + 1) begin : row_gen
    for (j = 0; j < SYS_SIZE; j = j + 1) begin : col_gen
      // compute cell type
      if (j < SYS_SIZE - i - 1) begin
        // PE_EMPTY: no instantiation
      end
      else if (i >= SA_PIXEL_WIDTH-1 && j <= i && j >= SYS_SIZE - i - 1 && j >= (KERNEL_SIZE - 1) + (i -
(SA_PIXEL_WIDTH - 1))) begin
        // PE_MAC
        processing_element #( ) //instantiation . . .
      end
      else begin
        // PE_DELAY [DE]
        delay_element #( ) //instantiation . . .
      end
    end
  end
end
endgenerate
```

- The above code summarizes the methodology that I have followed.
- This method can be applied to instantiation of wires and initialization of certain signals.

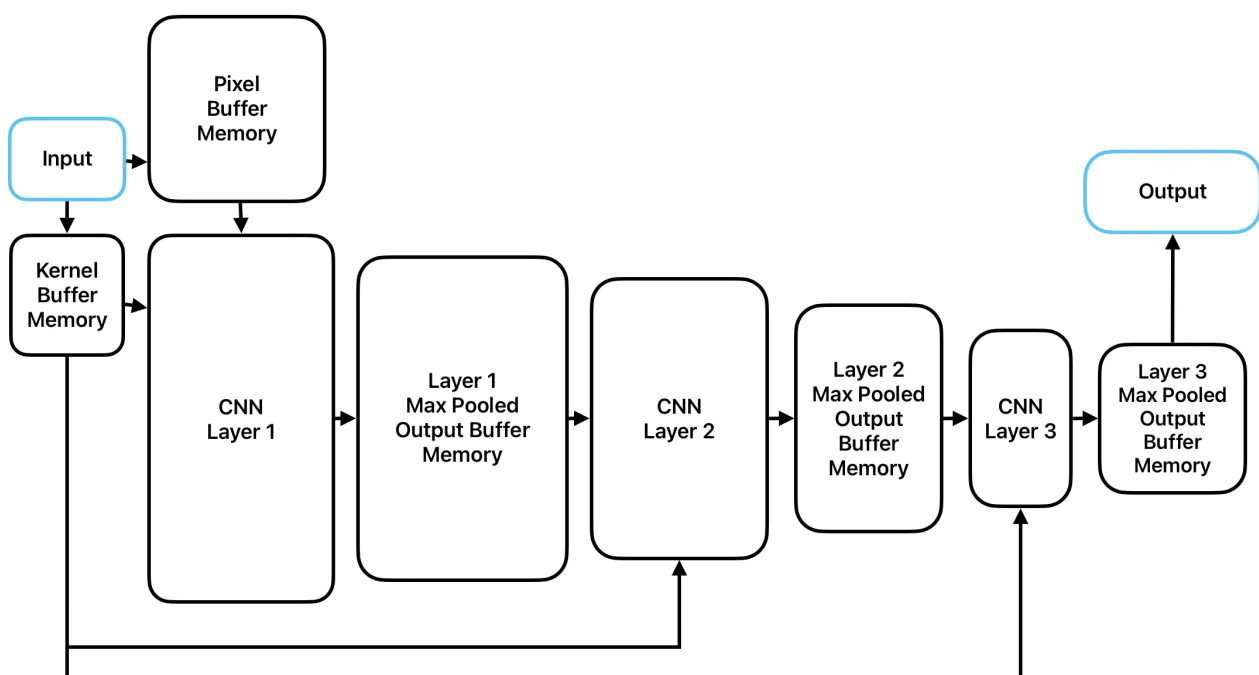
A Systolic array and continuous parallel max-pooling without sliding window buffers (windowless) was chosen for the proposed objectives.

Motivation :

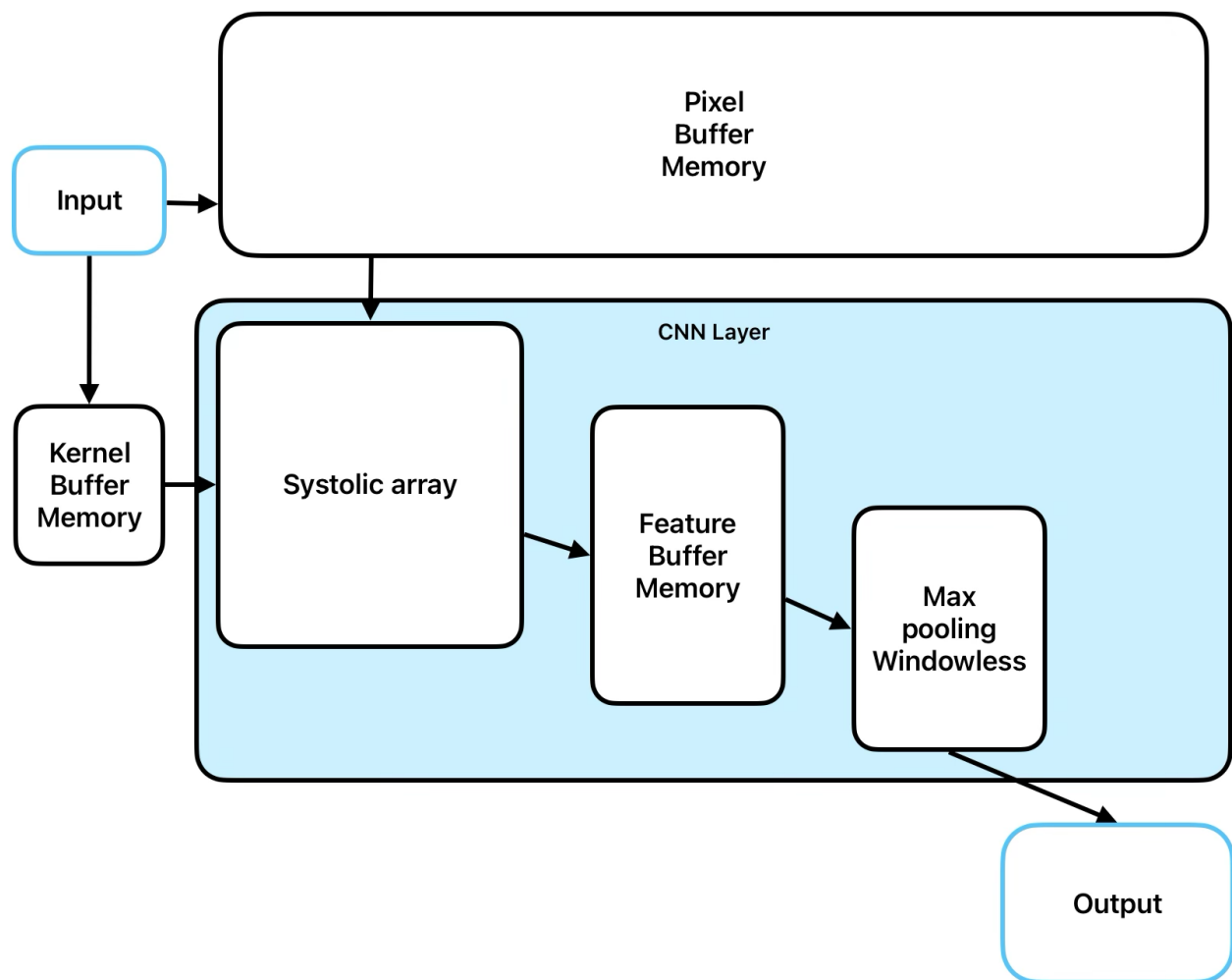
There are Two operations which demand lots of computational power, costing multiply-accumulate modules, MUXes for comparators, and lots of IO bandwidth, buffer memory, and clock cycles. Those operations are Convolution and Max-Pooling.

The given data flow sketches and the descriptions elaborate on why systolic arrays and continuous max-pooling without sliding window buffers was chosen, along with why specific decisions were made.

Data Flow Sketches and explanations:



3 Layer CNN Accelerator Model used in this project



Structure of a modular layer

As we can observe, each layer is modular, so it can be used to construct multiple layers. The first layer shall be connected directly to the input buffer memories. The last layer's max pooling buffer shall be connected as the final data output buffer.

Intermediate layers use previous layer's output as input, and they output the input of the next layer and so on.

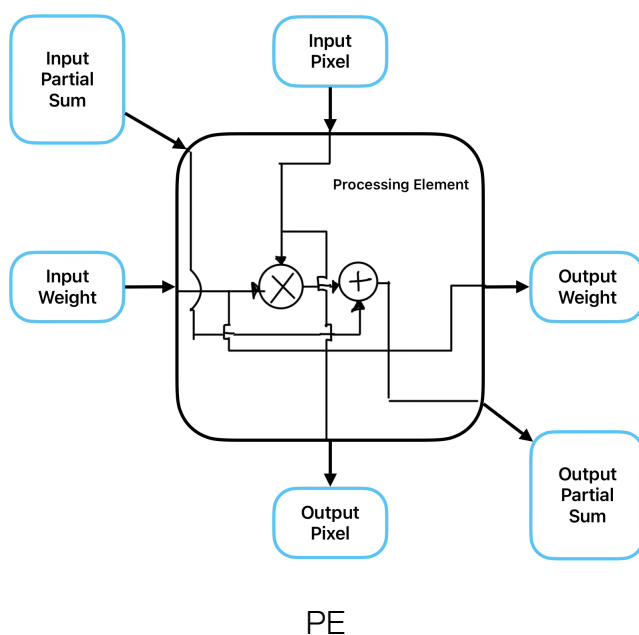
This project uses 3 Layer CNN system to showcase the modularity and the processing ability. Within each layer, there is systolic array for convolution, whose output is written to a feature buffer map, which is read by the max pooler continuously, which in turn, is written to output buffer.

Systolic arrays operate rhythmically, where data is continuously flowing, responding to the clock cycle and is synchronized. The version presented above is tailored for convolution, where the inputs should be staggered. *If there exists an input element matrix of n rows and m columns, and a kernel element matrix of u rows and v columns, the systolic array shall be of dimension $n \times u$.*

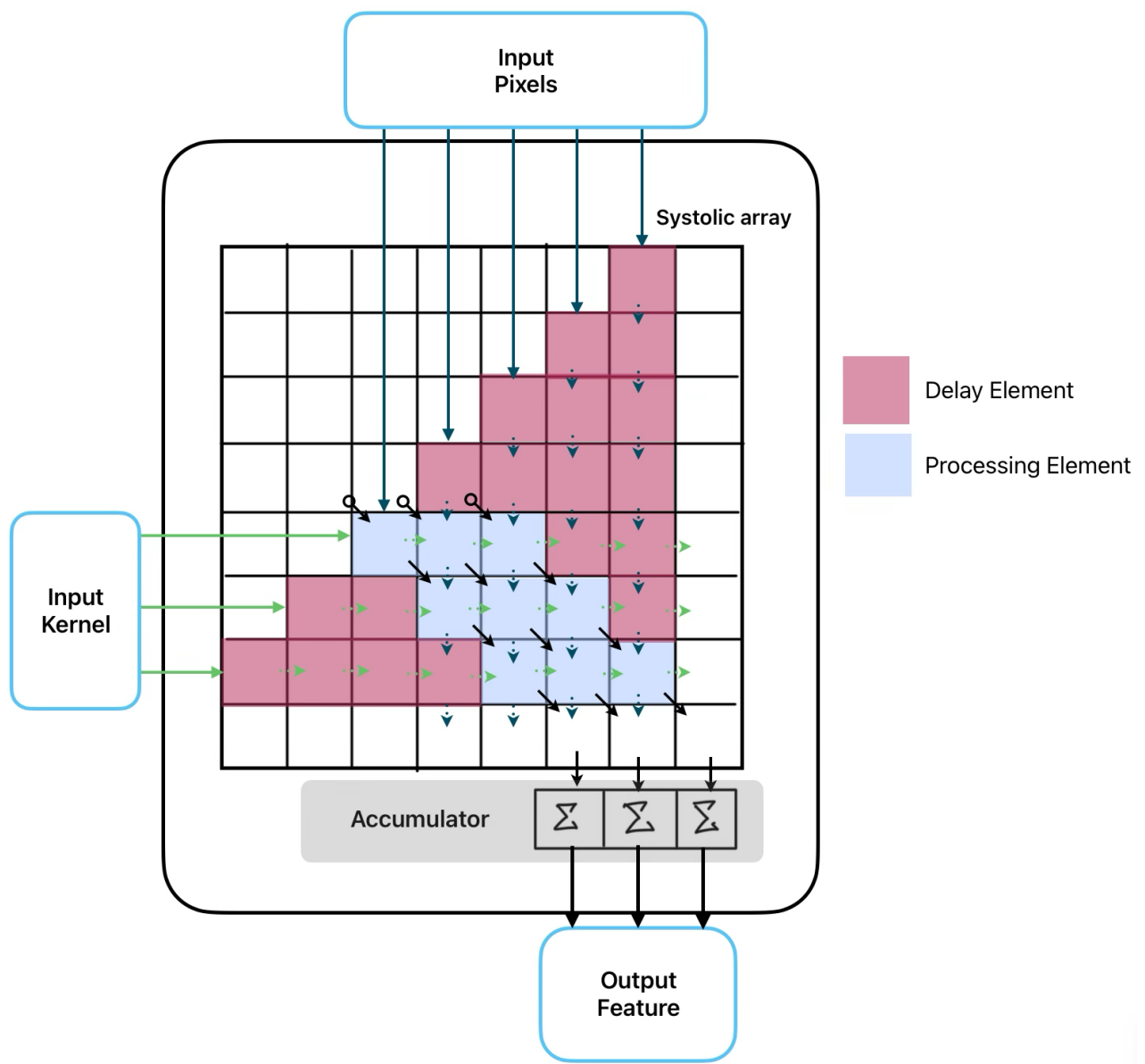
Note that the buffer memories are arranged in such a way that read/write addresses actually addresses the column number/index for easy dataflow and control logic.

The buffer memories are just multiple instantiation (rows) of line buffer memory, which has both read and write port (dual port, independent). So row number is just the row number of the memory array.

The data flows only when the clock advances, and the array is enabled.



The processing element [PE] takes in the input pixel/feature from the top, input kernel weight from the left, partial sum from top left, every clock cycle, from neighboring element / direct input.

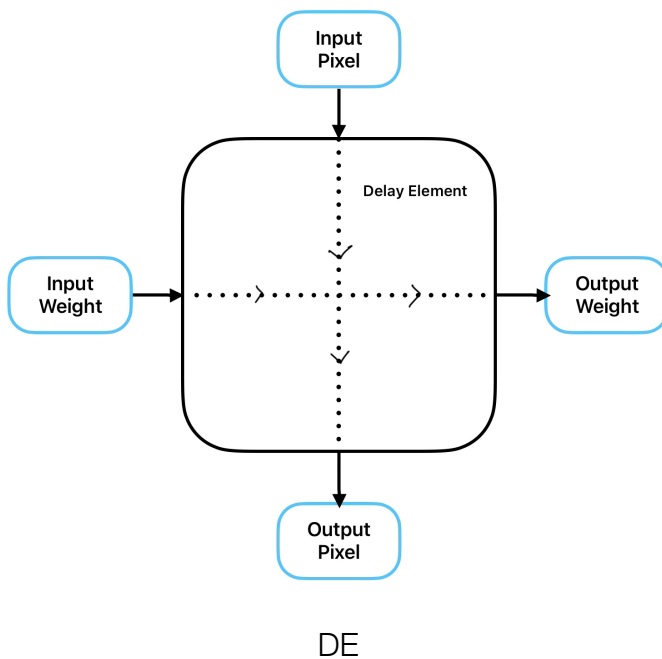


Systolic Array 5x3

Within the next clock cycle, the PE does the MAC [multiply accumulate] operation where $\text{partial sum out} = \text{weight} * \text{input} + \text{partial sum in}$.

In the next clock cycle, while the inputs are being taken as usual, the outputs are produced, where right side is kernel, bottom is pixel/feature, bottom right is the partial sum out.

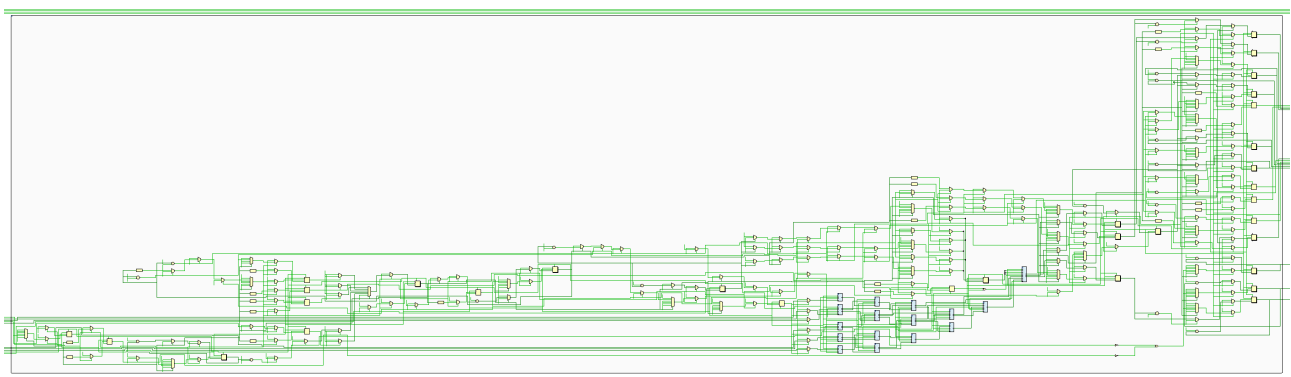
Diagonal partial sum propagation has been used.



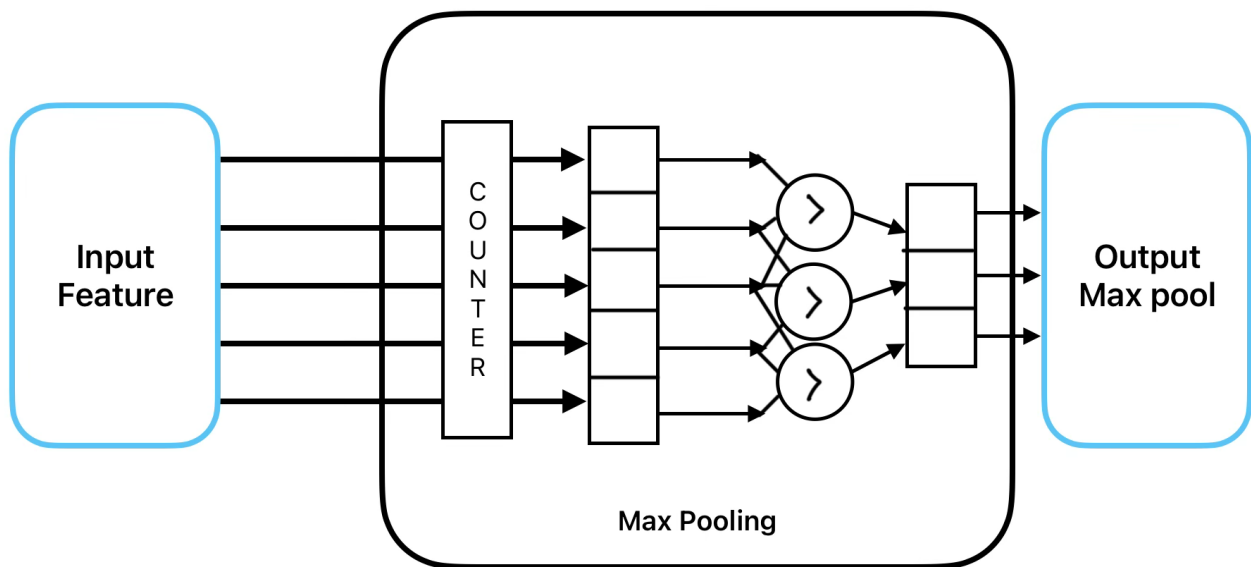
Delay Elements [DE] just propagate the inputs mirroring the PE, just without any MAC operation, and no partial sum propagation.

At the bottom, where the array ends, we have the accumulator, which precisely only accumulates the partial sums within the "window", which in turn outputs the feature map. Addressing and control of timing is done by a rather complex control logic and flags. ReLU is just a simple comparator at the end of the accumulator. Where it clips any value below zero to ground/zero.

This makes up the systolic array, enables data reuse, reduces IO, and introduces parallel processing, reduces clock cycles when used at an optimal size.



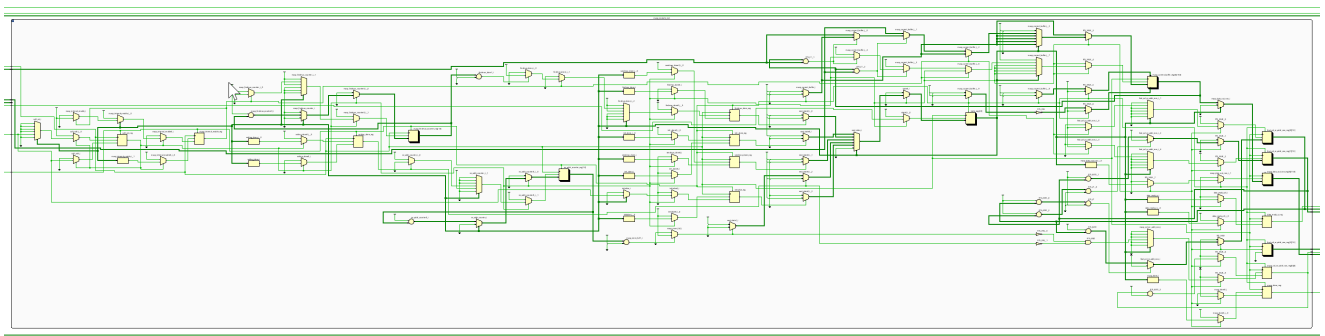
even a small systolic array has complicated control logic



Max pooling module :

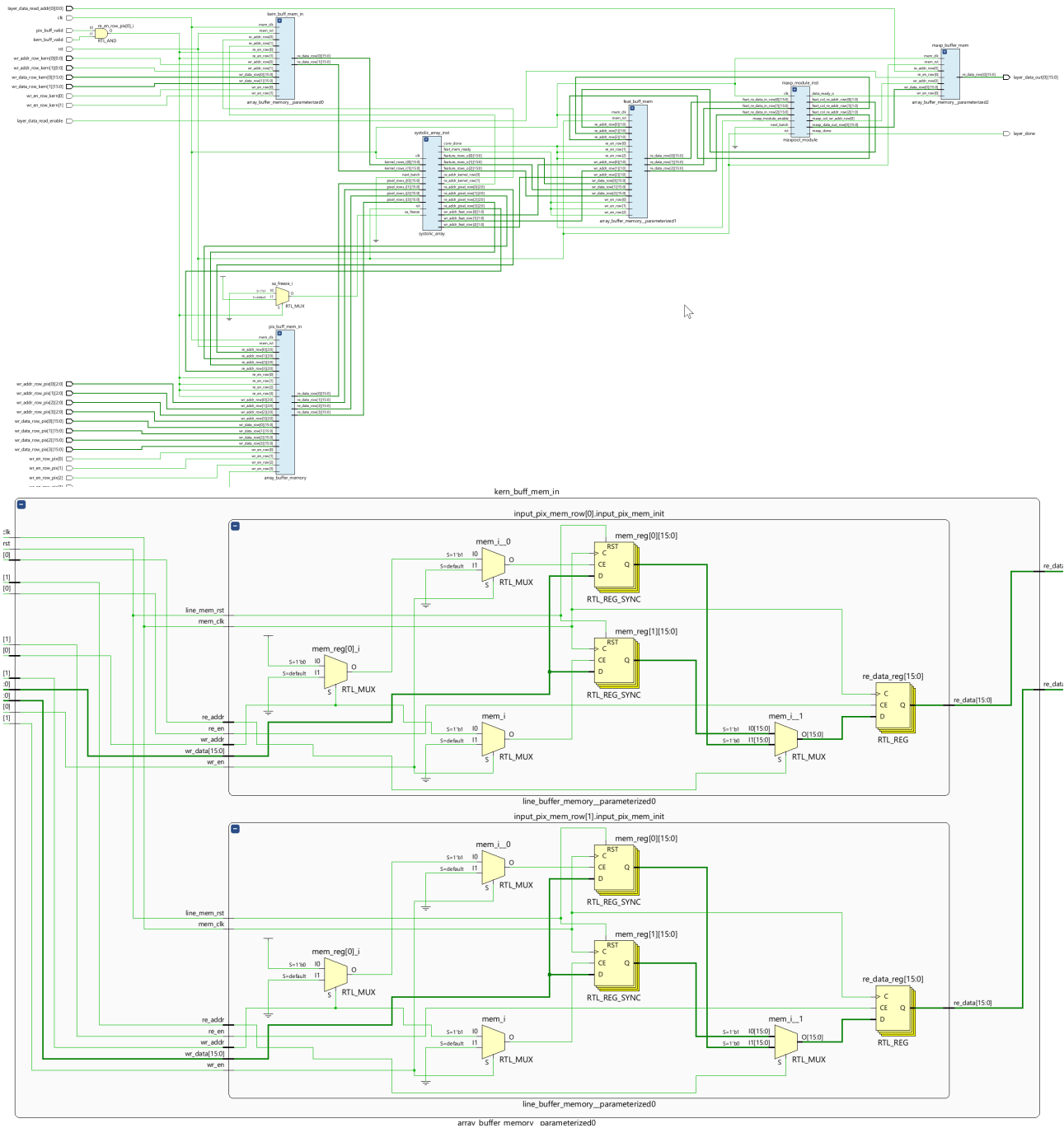
For windowless operation, we just take "POOL_SIZE" amount of columns streamed one column at a time, compare them every cycle, along with appropriate STRIDE taken, and determine the maximum value in the pool size. And after POOL_SIZE amount of columns are over, we output/write it to max pool buffer, and continue till end.

The windowless operation is totally based on counters and control logic.



windowless max pooling gets more effective as size increases

RTL Elaboration of a sample scaled down single layer:



Dual Port Buffer memory elaboration for reference

Name	Value
clk	1
rst	0
wr_en_row_pix[0:11]	0,0,0,0,0,0,0,0,0,0,0,0
wr_addr_row_pix[0:11][3:0]	b,b,b,b,b,b,b,b,b,b,b,b
wr_data_row_pix[0:11][15:0]	025d,0294,02cb,0302,0339,0370,03a7,03de,0415,044c,0483,04ba
pix_buff_valid	1
wr_en_row_kern[0:2]	0,0,0
wr_addr_row_kern[0:2][1:0]	2,2,2
wr_data_row_kern[0:2][15:0]	0339,0339,0339
kern_buff_valid	1
layer1_data_read_enable	0
layer1_data_out_addr[0:4][2:0]	4,4,4,4,4
layer1_data_out[0:4][15:0]	0092,00ab,00c3,00db,00f5
layer_done	1
SA_PIXEL_WIDTH[31:0]	0000000c
KERNEL_SIZE[31:0]	00000003
PIX_RAM_DEPTH[31:0]	0000000c
BIT_WIDTH[31:0]	00000010
MAX_POOL_STRIDE[31:0]	00000002
MAX_POOL_SIZE[31:0]	00000002
FEAT_PIX_WIDTH[31:0]	0000000a
FEAT_RAM_DEPTH[31:0]	0000000a
MAXP_PIX_WIDTH[31:0]	00000005
MAXP_RAM_DEPTH[31:0]	00000005

[illegible]

3 Layer CNN system - test-bench run

Conclusion :

- Convolution operation with systolic array facilitating parallel processing and data reuse was done.
- Integrate ReLU activation function, by appending the feature map output with a simple comparator
- Made max-pooling module without sliding window buffers, and with stride and pool size settings.
- Connecting the modules together and form a fully working, functional 3 Layer CNN for establishing proof of concept, proving modularity and proper parametrization for scalability.
- All of the required objectives were fulfilled.

Concerns, Potential Improvements, and Future Scope :

- **Pipelining :**
- While parallel processing, data reuse, IO reduction was done, clock cycles could be reduced drastically especially if the inputs images are continuous (videos/tiled images), using pipelining.
- Pipelining can be carried out with precise timing within the layer.
- Provisions were made for future pipelining improvements in the HDL code.
- Larger images can be tiled and then processed tile-wise, we can improve the efficiency by doing so and combining pipelining.

- By pipelining : we have two options, and both can be carried out :
 - Layer Level pipelining : within the layer itself, the systolic array and max pooling module can be pipelined, currently the max pooling waits for the completely full feature map.
 - System Level pipelining : Within the entire system, each layer can be pipelined, currently, only one layer operates at a time, since the purpose was to demonstrate systolic arrays and scalability.

- **Bounds :**

- This system is only meaningful when the size of images are within reasonable bounds, but there are provisions to tile them.
- When the image inputs are too small, a normal naive CNN would outperform this. Depending on the application, the lower bound of size shall be decided.
- When the image inputs are selected at the optimal size, the savings made by the systolic array and max pooling become visible.
- The upper bound is only constrained by the amount of resources available on the target device, naturally.

- **Regarding RAM :**

- I have used Dual Port Line Buffer Memory Arrays (synchronous) for the best possible way to stream in inputs within the system.

- Ping pong buffer system , where dual buffers are maintained, can be used for the input pixels, so that when pipelining is applied, the input pixels are always ready.

- **Regarding IO :**

- While a large reduction in IO has been made, DMA system has been left out of demo. A DMA would stream word by word to the input buffers. Currently the test bench emulates the DMA partially.
- Using the DMA - AXI4 system would drastically reduce IO ports required, and makes sure inputs are valid and outputs are properly registered.
- The design has been made assuming the MAC operation is done in one clock cycle or lesser time. Similarly, the RAM has been assumed to have one clock cycle delay to both read and write. In practice, real world situation, the true delay has to be accounted for.

- **Data Type :**

- A Q0.15, 16 Bit , signed fixed point system has been used for this application, since the system is modular, modifying the MAC module within the PE, and the accumulator module is enough.